

But can this happen in a machine learning application for real? Yes! Imagine a situation where you are training your model with an inadequate or corrupted data set. This is just one of the many reasons why both these learning models, supervised and unsupervised, are still thriving in the fields of AI and machine learning. In the coming articles in this series, we will treat these topics more formally. Now let us learn to use JupyterLab, a very powerful tool available for developing AI based applications.

Introduction to JupyterLab

In the previous articles in this series, we were using the Linux terminal to run Python code for simplicity. Now, it is time to introduce yet another powerful player in the world of AI, JupyterLab. Remember, in the first article in this series we considered a few other alternatives to JupyterLab. But after considering various pros and cons of each and every one of them, we have decided to stick with JupyterLab. As JupyterLab is more powerful than Jupyter Notebook, our choice is further justified. There are many reasons for preferring JupyterLab over the Linux terminal while developing AI based applications. For example, the library and packages we need to use are often available by default in JupyterLab. Another reason is the ease of collaboration with a group of programmers or researchers. There are many other reasons too, and we will explore these in due course.

We have already learned how to install JupyterLab in the first article in this series. Assuming you have installed JupyterLab in your system, with the instructions given earlier, use the command `jupyter lab` or `jupyter-lab` to open JupyterLab in your default browser, like Mozilla Firefox, Google Chrome, etc. Figure 1 shows the launcher of the JupyterLab (just a part of the window) opened in my default browser. A Python shell called

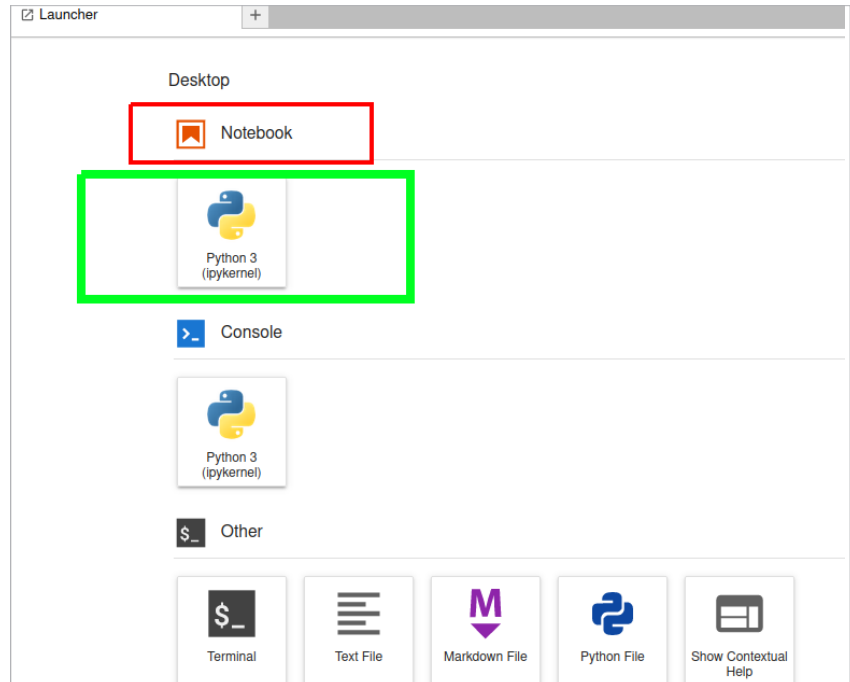


Figure 1: The launcher of the JupyterLab

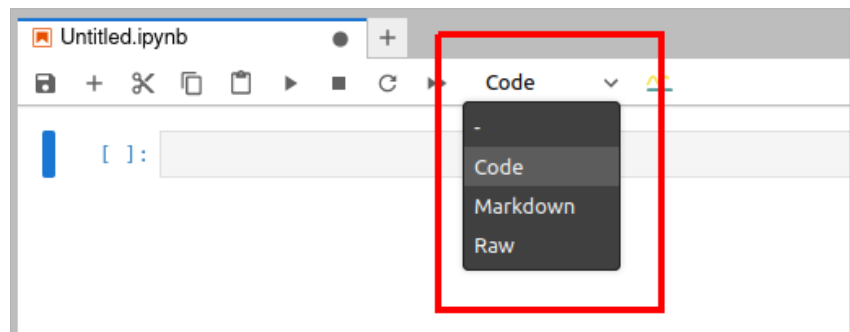


Figure 2: A Jupyter Notebook window

IPython (Interactive Python) is used in JupyterLab. But IPython has an independent existence and can be executed from your Linux terminal with the command `ipython`.

For the time being, we will start learning JupyterLab by using Jupyter Notebook alone, just one of the many features available with JupyterLab. In Figure 1, the button to be clicked to open a Jupyter Notebook is marked with a green rectangle. On clicking this button, you may be asked to select the kernel. If you have followed the installation steps given in this series, the sole choice would be Python 3

(`ipykernel`). But note that you can install kernels for programming languages like C++, R, MATLAB, Julia, etc, and many more in JupyterLab. Indeed, the complete list is quite large and can be found at <https://github.com/jupyter/jupyter/wiki/Jupyter-kernels>.

Now let us try to understand the working of a Jupyter Notebook very quickly. Figure 2 shows a Jupyter Notebook window. Notice that the extension of the open Jupyter Notebook file is `.ipynb`.

In Figure 2, you can see three different options, 'code', 'markdown', and 'raw' (marked with a red